

EVALUASI TENGAH SEMESTER PEMROGRAMAN KONTROLLER

BAGIAN B : STUDI LITERATUR & SIMULASI METODE BARU

Dosen Pengampu : Ahmad Radhy, S.Si., M.Si.



Oleh:

Annora Prima Prasista 2042241046

Naila Sa'adatul Rozyidah 2042241114

PRODI D4 TEKNOLOGI REKAYASA INSTRUMENTASI
DEPARTEMEN TEKNIK INSTRUMENTASI
FAKULTAS VOKASI
INSTITUT TEKNOLOGI SEPULUH NOPEMBER
2026

DAFTAR ISI

1	Pendahuluan	2
1.1	Latar Belakang	2
1.2	Rumusan Masalah	3
1.3	Tujuan	3
2	Studi Literatur	4
2.1	Ringkasan Jurnal Acuan	4
2.2	Daftar Future Work	7
3	Perancangan Sistem	9
3.1	Judul Metode	9
3.2	Diagram Blok	10
3.3	Penjelasan Diagram Blok	10
3.3.1	Penjelasan Diagram Blok Sistem	10
3.3.2	Kesimpulan	11
3.4	Flowchart Sistem	12
3.5	Penjelasan Flowchart	12
3.6	Deskripsi Metode	13
4	Simulasi dan Hasil	15
4.1	Penjelasan Rangkaian Sistem	15
4.1.1	ESP32-S3	15
4.1.2	Sensor Ultrasonik HC-SR04	15
4.1.3	LED Hijau	16
4.1.4	LED Kuning	16
4.1.5	LED Merah	16
4.1.6	Koneksi Kabel	16
4.2	Analisis Grafik Traffic Monitoring	17
4.3	Grafik Jarak Kendaraan vs Waktu	18
4.4	Grafik Durasi Lampu Hijau vs Waktu	19
4.5	Grafik Status Lampu vs Waktu	20
5	Keunggulan Metode	21
6	Dokumentasi GitHub dan Publikasi LinkedIn	22
6.1	Repository GitHub	22
6.2	LinkedIn	22
7	Kesimpulan	23

BAB 1 Pendahuluan

1.1 Latar Belakang

Perkembangan teknologi embedded system dan Internet of Things (IoT) saat ini semakin banyak yang diterapkan pada berbagai bidang salah satunya sistem transportasi pintar atau intelligent transportation system. Salah satu permasalahan yang masih terjadi pada sistem lalu lintas konvensional adalah penggunaan traffic light dengan metode fixed timer yaitu durasi lampu lalu lintas yang tetap tanpa memperhatikan kondisi kendaraan di jalan. Sistem ini sering menyebabkan kurangnya efisien terutama ketika kondisi menjadi lebih panjang karena durasi lampu hijau tidak menyesuaikan tingkat kepadatan kendaraan secara real-time.

Untuk mengatasi permasalahan tersebut diperlukan sistem pengendali lampu lalu lintas yang lebih adaptif dan mampu menyesuaikan durasi lampu berdasarkan kondisi kendaraan di jalan. Sistem adaptive traffic light controller menjadi salah satu solusi karena dapat melakukan monitoring kondisi lalu lintas secara real-time menggunakan sensor dan mikrokontroler. Dengan sistem ini pengaturan waktu lampu lalu lintas menjadi lebih fleksibel sehingga diharapkan dapat mengurangi antrean kendaraan dan meningkatkan efisiensi lalu lintas.

Pada metode kali ini menggunakan sensor ultrasonik HC-SR04 sebagai sensor utama untuk mendeteksi jarak kendaraan atau panjang antrean kendaraan pada persimpangan jalan. Data dari sensor kemudian diproses menggunakan mikrokontroler ESP32-S3 yang berfungsi sebagai embedded controller utama. ESP32-S3 dipilih karena memiliki performa yang baik untuk embedded system, mendukung komunikasi IoT, serta mampu bekerja pada sistem real-time dengan resource yang terbatas. Selain itu, metode menggunakan bahasa program Rust sebagai pengendali sistem embedded. Rust dipilih karena memiliki keunggulan memory safety, concurrency safety, dan efisiensi penggunaan resource dibandingkan bahasa pemrograman embedded konvensional seperti C atau C++. Penggunaan Rust pada embedded system diharapkan dapat meningkatkan stabilitas dan keamanan sistem, terutama dalam mengurangi risiko error seperti buffer overflow dan kesalahan akses memori.

Metode yang diusulkan dalam penelitian ini yaitu Adaptive Resource-Aware Traffic Light Controller with Safety Mode Using Rust on ESP32-S3 and Ultrasonic Sensor. Sistem ini menggabungkan konsep adaptive timing, real-time monitoring, resource-aware processing, serta safety mode dalam satu sistem terintegrasi. Ketika kendaraan terdeteksi padat, sistem akan memperpanjang durasi lampu hijau agar antrean kendaraan dapat berkurang lebih cepat. Sebaliknya, ketika kondisi jalan normal, sistem menggunakan durasi lampu standar sehingga pengaturan lalu lintas menjadi lebih efisien.

Selain itu, sistem juga dilengkapi dengan safety mode untuk mengantisipasi ter-

jadinya gangguan pembacaan sensor atau error pada sistem. Dengan adanya fitur ini, sistem diharapkan dapat bekerja lebih aman, stabil, dan adaptif pada kondisi lalu lintas yang berubah secara dinamis. Oleh karena itu, penelitian ini diharapkan dapat menjadi salah satu pengembangan intelligent transportation system berbasis embedded system dan IoT yang lebih modern, efisien, dan aman.

1.2 Rumusan Masalah

1. Bagaimana merancang sistem adaptive traffic light controller berbasis embedded system menggunakan ESP32-S3?
2. Bagaimana cara implementasi bahasa Rust pada ESP32-S3 untuk pengendalian traffic secara real-time?
3. Bagaimana sistem mendeteksi kepadatan kendaraan menggunakan sensor ultrasonic HC-SR04?
4. Bagaimana cara menerapkan adaptive timing dan safety mode pada sistem traffic light agar mampu menyesuaikan kondisi lalu lintas secara otomatis?

1.3 Tujuan

1. Merancang dan membangun sistem *adaptive traffic light controller* berbasis *embedded system* menggunakan ESP32-S3.
2. Mengimplementasikan bahasa pemrograman Rust pada ESP32-S3 untuk pengendalian *traffic light* secara *real-time*.
3. Mengimplementasikan sensor ultrasonik HC-SR04 untuk mendeteksi kepadatan kendaraan secara *real-time*.
4. Menerapkan *adaptive timing* dan *safety mode* pada sistem *traffic light* agar mampu menyesuaikan kondisi lalu lintas secara otomatis.

BAB 2 Studi Literatur

2.1 Ringkasan Jurnal Acuan

No	Judul	Penulis	Tahun	Metode	Future Work
1	MicroFlow: An Efficient Rust-Based Inference Engine for TinyML	Matteo Canelos et al.	2025	Compiler-based TinyML inference engine berbasis Rust dengan static memory allocation dan paging memory.	Pengembangan dukungan neural network lebih kompleks dan optimasi arsitektur MCU lain.
2	Rust for Embedded Systems: Current State and Open Problems	Ayushi Sharma et al.	2024	Systematic study embedded Rust crates dan interoperability Rust-C.	Pengembangan tools isolasi unsafe code dan support MCU non-ARM.
3	Embedded Sensor Data Fusion and TinyML for Real-Time Remaining Useful Life Estimation of UAV Li Polymer Batteries	Jutarut Chao-raingern et al.	2025	TinyML berbasis FFNN dengan embedded sensor fusion.	Pengembangan model LSTM dan optimasi TinyML untuk resource terbatas.
4	Energy-Efficient AI on the Edge	Nicolas Witt et al.	2024	AutoML, deep compression, dan Bayesian optimization untuk embedded AI.	Pengembangan neuromorphic computing dan ultra low power AI.
5	A Hard Real-Time Kernel for CPIoT Systems With Safety Features in Rust	Nayancy Gupta et al.	2025	Kernel hard real-time berbasis Rust untuk sistem CPIoT.	Peningkatan portability dan otomatisasi HAL generation.

No	Judul	Penulis	Tahun	Metode	Future Work
6	An Accurate Real-Time Object Tracking Method for Resource-Constrained Devices	A. Sardaryan et al.	2024	Hybrid object tracking berbasis correlation filter dan transformer.	Optimasi tracking system untuk embedded resource-constrained devices.
7	Quantization and Deployment of Deep Neural Networks on Microcontrollers	Pierre-Emmanuel Novac et al.	2021	Quantization 8-bit dan 16-bit untuk deployment deep learning pada microcontroller.	Optimasi inference engine dan deployment CNN pada embedded AI.
8	Dependability in Embedded Systems: A Survey of Fault Tolerance Methods and Software-Based Mitigation Techniques	Mohammadreza Amel Solouki et al.	2024	Fault tolerance berbasis hardware dan software redundancy.	Adaptive fault tolerance berbasis AI dan peningkatan reliability embedded systems.
9	Advancing TinyML in IoT: A Holistic System-Level Perspective for Resource-Constrained AI	Leandro Antonio Pazmiño Ortiz et al.	2025	Systematic review TinyML pada IoT dan embedded AI.	On-device training dan federated learning untuk TinyML.
10	Pervasive AI for IoT Applications: A Survey on Resource-Efficient Distributed Artificial Intelligence	Emna Baccour et al.	2022	Distributed learning dan federated learning pada IoT systems.	Adaptive pervasive inference dan intelligent resource scheduling.

No	Judul	Penulis	Tahun	Metode	Future Work
11	Composite Human Activity Recognition Utilizing Knowledge Distillation and Sensor Fusion Focusing on Resource Constrained Microcontrollers	Athar Noor Mohammad Rafee et al.	2026	TinyML berbasis knowledge distillation dan sensor fusion pada ESP32-S3.	Optimasi latency, konsumsi daya, dan lightweight TinyML architecture.
12	ESP-Fi HAR: A Low-Power WiFi CSI Dataset for Ad-Hoc IoT Human Activity Recognition	Zhiwei Wen et al.	2026	Human Activity Recognition berbasis WiFi CSI menggunakan ESP32-C3.	Pengembangan collaborative sensing dan multi-user HAR pada AIoT.
13	A Comprehensive Survey of TinyML-Based Biometric Recognition for IoT Edge Devices	Siham Es-sahraoui et al.	2026	Survey TinyML biometric recognition pada IoT edge devices.	Pengembangan federated learning dan multimodal biometric fusion.
14	Composite Human Activity Recognition Utilizing Knowledge Distillation and Sensor Fusion Focusing on Resource Constrained Microcontrollers	Athar Noor Mohammad Rafee et al.	2026	Sensor fusion dan lightweight CNN untuk embedded TinyML pada ESP32-S3.	Pengembangan scalable real-time Human Activity Recognition pada embedded AI.
15	Quantization and Deployment of Deep Neural Networks on Microcontrollers	Pierre-Emmanuel Novac et al.	2021	Framework MicroAI untuk quantization dan deployment deep neural network.	Pengembangan lightweight embedded AI architecture dan optimasi RAM allocator.

“latex id=”futureworklist”

2.2 Daftar Future Work

1. **MicroFlow: An Efficient Rust-Based Inference Engine for TinyML**
Pengembangan support operator neural network yang lebih kompleks, kompatibilitas model lain seperti ONNX, serta optimasi TinyML pada lebih banyak arsitektur micro-controller resource-constrained.
2. **Rust for Embedded Systems: Current State and Open Problems**
Pengembangan interoperabilitas Rust-C yang lebih baik, peningkatan support MCU non-ARM, optimasi *no_std configuration*, serta pengembangan isolasi *unsafe code* pada embedded Rust systems.
3. **Embedded Sensor Data Fusion and TinyML for Real-Time Remaining Useful Life Estimation of UAV Li Polymer Batteries**
Pengembangan TinyML architecture yang lebih kompleks seperti LSTM serta optimasi performa embedded AI pada platform dengan resource terbatas.
4. **Energy-Efficient AI on the Edge**
Pengembangan hardware-aware optimization, *on-device learning*, neuromorphic computing, serta ultra low power embedded AI untuk edge devices.
5. **A Hard Real-Time Kernel for CPIOt Systems With Safety Features in Rust**
Pengembangan file-system support, otomatisasi HAL generation, serta peningkatan portability kernel Rust untuk berbagai microcontroller embedded systems.
6. **An Accurate Real-Time Object Tracking Method for Resource-Constrained Devices**
Pengembangan object tracking yang lebih efisien dan optimal pada embedded edge devices dengan resource terbatas.
7. **Quantization and Deployment of Deep Neural Networks on Microcontrollers**
Pengembangan optimasi inference engine, allocator RAM, serta deployment lightweight AI architecture pada embedded microcontroller systems.
8. **Dependability in Embedded Systems: A Survey of Fault Tolerance Methods and Software-Based Mitigation Techniques**
Pengembangan adaptive fault tolerance berbasis AI, hybrid fault mitigation, serta peningkatan reliability dan control-flow protection pada embedded real-time systems.
9. **Advancing TinyML in IoT: A Holistic System-Level Perspective for Resource-Constrained AI**
Pengembangan dynamic model adaptation, *on-device learning*, federated learning, serta resilience system untuk TinyML pada embedded IoT devices.
10. **Pervasive AI for IoT Applications: A Survey on Resource-Efficient Distributed Artificial Intelligence**

Pengembangan adaptive pervasive inference, intelligent resource scheduling, privacy-preserving AI, serta distributed intelligence untuk IoT embedded systems.

11. **Composite Human Activity Recognition Utilizing Knowledge Distillation and Sensor Fusion Focusing on Resource Constrained Microcontrollers**

Pengembangan lightweight TinyML architecture dengan latency lebih rendah, optimasi sensor fusion, serta pengurangan konsumsi daya pada ESP32-S3 embedded AI systems.

12. **ESP-Fi HAR: A Low-Power WiFi CSI Dataset for Ad-Hoc IoT Human Activity Recognition**

Pengembangan collaborative sensing, multi-user HAR, edge-to-cloud architecture, dan domain adaptation untuk AIoT system berbasis ESP32.

13. **A Comprehensive Survey of TinyML-Based Biometric Recognition for IoT Edge Devices**

Pengembangan federated learning, Tiny AutoML, multimodal biometric fusion, serta peningkatan robustness dan energy efficiency pada TinyML edge systems.

14. **Composite Human Activity Recognition Utilizing Knowledge Distillation and Sensor Fusion Focusing on Resource Constrained Microcontrollers**

Pengembangan scalable real-time Human Activity Recognition berbasis sensor fusion dan TinyML pada embedded edge AI systems dengan resource terbatas.

15. **Quantization and Deployment of Deep Neural Networks on Microcontrollers**

Pengembangan lightweight embedded AI architecture, optimasi quantization, serta peningkatan power efficiency dan latency pada resource-constrained edge devices.

BAB 3 Perancangan Sistem

3.1 Judul Metode

Adaptive Resource-Aware Traffic Light Controller with Safety Mode Using Rust on ESP32-S3 and Ultrasonic Sensor

Metode yang diusulkan pada penelitian ini merupakan sistem *adaptive traffic light controller* berbasis *embedded system* menggunakan mikrokontroler ESP32-S3, sensor ultrasonik HC-SR04, dan bahasa pemrograman Rust sebagai pengendali utama sistem. Metode ini dirancang untuk mengatur durasi lampu lalu lintas secara otomatis berdasarkan kondisi kepadatan kendaraan yang dibaca secara *real-time*.

Sistem bekerja dengan memanfaatkan sensor ultrasonik HC-SR04 untuk mendeteksi jarak kendaraan atau panjang antrean kendaraan pada persimpangan jalan. Data hasil pembacaan sensor kemudian diproses oleh ESP32-S3 menggunakan algoritma *adaptive timing* untuk menentukan kondisi lalu lintas. Ketika kendaraan terdeteksi padat atau jarak kendaraan terlalu dekat dengan sensor, sistem akan mengaktifkan *High Traffic Safety Mode* dengan memperpanjang durasi lampu hijau agar antrean kendaraan dapat berkurang lebih efektif. Sebaliknya, ketika kondisi jalan relatif lancar, sistem akan menggunakan durasi lampu normal sehingga pengaturan lalu lintas menjadi lebih efisien.

Selain menerapkan *adaptive timing*, metode ini juga dilengkapi dengan *safety mode* sebagai mekanisme keamanan apabila terjadi kesalahan pembacaan sensor atau gangguan pada sistem *embedded*. Dalam kondisi tertentu, sistem dapat mengaktifkan mode keamanan seperti lampu kuning berkedip atau seluruh lampu berubah menjadi merah sementara untuk mengurangi potensi konflik lalu lintas.

Konsep *resource-aware system* pada metode ini diterapkan dengan cara mengoptimalkan penggunaan memori, waktu pemrosesan, serta penggunaan fitur-fitur penting pada ESP32-S3 sesuai kebutuhan sistem. Dengan demikian, sistem dapat bekerja lebih ringan, responsif, dan stabil meskipun dijalankan pada perangkat *embedded* dengan resource terbatas.

Penggunaan bahasa pemrograman Rust pada ESP32-S3 dipilih karena memiliki keunggulan seperti *memory safety*, *ownership system*, dan *concurrency safety* yang mampu mengurangi risiko kesalahan akses memori dibandingkan bahasa pemrograman *embedded* konvensional seperti C/C++. Oleh karena itu, metode ini diharapkan dapat menjadi salah satu pengembangan *intelligent transportation system* berbasis *embedded system* dan IoT yang lebih adaptif, aman, dan efisien.

3.2 Diagram Blok

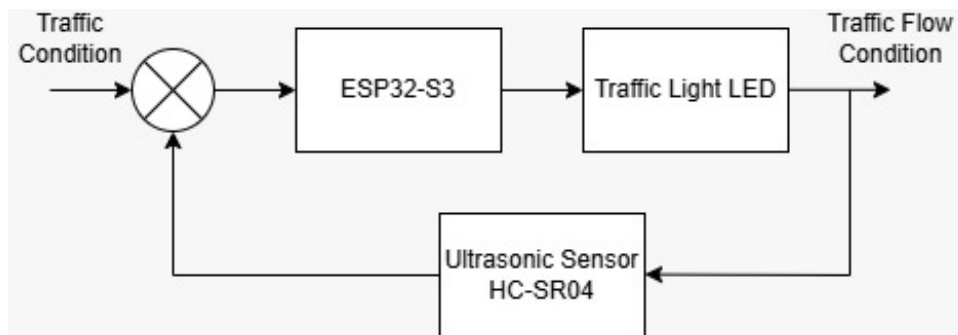


Figure 3.1: Diagram Blok Sistem

3.3 Penjelasan Diagram Blok

3.3.1 Penjelasan Diagram Blok Sistem

Diagram blok tersebut menunjukkan sistem *Adaptive Traffic Light Controller* berbasis ESP32-S3 dan sensor ultrasonik HC-SR04 untuk mengatur lampu lalu lintas berdasarkan kondisi kendaraan di jalan.

1. Traffic Condition (Kondisi Lalu Lintas)

Bagian ini merupakan kondisi kendaraan di jalan, seperti jumlah kendaraan atau tingkat kepadatan lalu lintas. Kendaraan yang berada di depan sensor akan menjadi input awal sistem.

2. Ultrasonic Sensor HC-SR04

Sensor HC-SR04 berfungsi mendeteksi keberadaan kendaraan dengan mengukur jarak objek menggunakan gelombang ultrasonik. Sensor memancarkan gelombang suara ultrasonik, kemudian menerima pantulan gelombang dari kendaraan. Dari waktu pantulan tersebut, sensor menghitung jarak kendaraan.

Data jarak ini digunakan untuk mengetahui kondisi lalu lintas:

- Jika jarak kendaraan dekat, maka jalan dianggap padat.
- Jika jarak kendaraan jauh atau tidak ada objek, maka jalan dianggap sepi.

3. ESP32-S3

ESP32-S3 bertindak sebagai pusat pengendali sistem. Mikrokontroler ini menerima data dari sensor HC-SR04, kemudian memproses informasi tersebut untuk menentukan kondisi lalu lintas.

Berdasarkan hasil pembacaan sensor:

- Jika kendaraan terdeteksi banyak atau padat, ESP32-S3 akan memperpanjang durasi lampu hijau.
- Jika kendaraan sedikit atau tidak ada, durasi lampu hijau dapat dipersingkat.

Dengan metode ini, sistem menjadi lebih adaptif dibandingkan sistem *fixed timer*.

4. Traffic Light LED

Output dari ESP32-S3 digunakan untuk mengontrol LED lampu lalu lintas yang terdiri dari:

- Lampu merah
- Lampu kuning
- Lampu hijau

LED akan menyala sesuai keputusan yang telah diproses oleh ESP32-S3 berdasarkan kondisi lalu lintas yang terdeteksi sensor.

5. **Traffic Flow Condition (Kondisi Arus Lalu Lintas)** Setelah lampu lalu lintas dikendalikan, kondisi arus kendaraan akan berubah. Jika lampu hijau menyala lebih lama saat jalan padat, maka antrean kendaraan dapat berkurang sehingga arus lalu lintas menjadi lebih lancar.

6. Sistem Umpan Balik (*Feedback*)

Diagram menunjukkan adanya jalur umpan balik dari kondisi arus lalu lintas menuju sensor HC-SR04. Artinya, sistem bekerja secara terus-menerus memantau kondisi jalan secara *real-time* sehingga pengaturan lampu lalu lintas dapat selalu menyesuaikan keadaan kendaraan di lapangan.

3.3.2 Kesimpulan

Sistem pada diagram blok ini bekerja dengan cara mendeteksi kepadatan kendaraan menggunakan sensor ultrasonik HC-SR04, kemudian ESP32-S3 memproses data tersebut untuk mengatur nyala lampu lalu lintas secara adaptif. Dengan metode ini, pengaturan lampu menjadi lebih efisien dan mampu mengurangi kemacetan dibandingkan sistem lampu lalu lintas konvensional yang menggunakan waktu tetap.

3.4 Flowchart Sistem

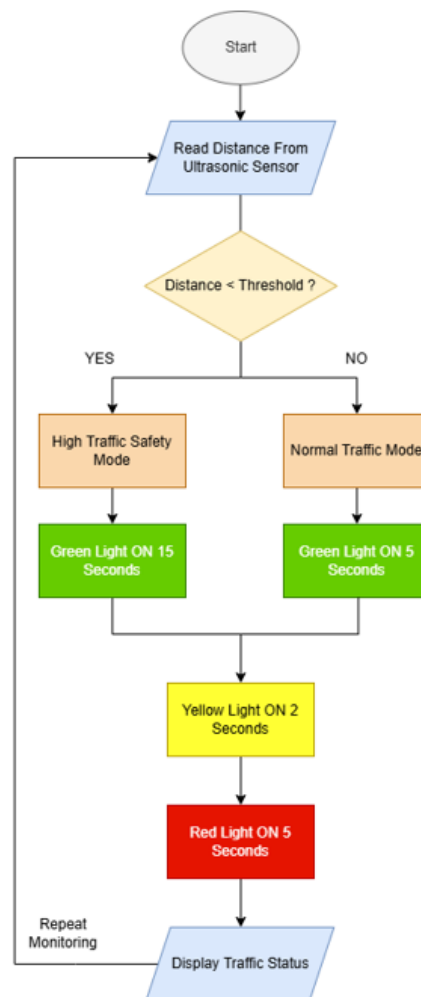


Figure 3.2: Flowchart

3.5 Penjelasan Flowchart

1. **Start**

Merupakan titik awal program saat sistem mikrokontroler dan sensor mulai dinyalakan serta diinisialisasi.

2. **Read Distance From Ultrasonic Sensor**

Sensor ultrasonik memancarkan gelombang untuk mengukur jarak kendaraan atau panjang antrean kendaraan pada lajur jalan sebagai data input utama sistem.

3. **Percabangan Kondisi (Distance dan Threshold)**

Data jarak dari sensor dievaluasi berdasarkan nilai ambang batas (*threshold*) yang telah ditentukan.

(a) **Jalur YES (Padat)**

Jika jarak kendaraan lebih kecil dari nilai *threshold*, maka antrean kendaraan dianggap padat atau rapat. Sistem akan masuk ke *High Traffic Safety Mode* dan menyalakan lampu hijau selama 15 detik agar antrean kendaraan dapat berkurang lebih efektif.

(b) **Jalur NO (Lancar)**

Jika jarak kendaraan lebih besar atau sama dengan nilai *threshold*, maka kondisi jalan dianggap lancar atau tidak padat. Sistem akan masuk ke *Normal Traffic Mode* dan menyalakan lampu hijau selama 5 detik untuk meningkatkan efisiensi waktu lampu lalu lintas.

4. **Fase Transisi (Yellow Light 2s)**

Setelah masa lampu hijau dari salah satu mode selesai, alur kembali menyatu menuju lampu kuning yang menyala selama 2 detik sebagai tanda transisi aman bagi pengendara untuk bersiap berhenti.

5. **Fase Penghentian (Red Light 5s)**

Lampu merah dinyalakan selama 5 detik untuk menghentikan lajur kendaraan tersebut dan memberikan kesempatan bagi lajur jalan lainnya.

6. **Display Traffic Status**

Sistem menampilkan informasi status kepadatan lalu lintas atau indikator mode sistem saat ini ke perangkat display seperti LCD atau serial monitor.

7. **Repeat Monitoring (Looping)**

Setelah satu siklus selesai, proses kembali menuju tahap pembacaan sensor ultrasonik. Proses ini dilakukan secara terus-menerus agar sistem lampu lalu lintas tetap adaptif terhadap perubahan kondisi jalan secara *real-time*.

3.6 Deskripsi Metode

Metode yang diusulkan pada penelitian ini yaitu *Adaptive Resource-Aware Traffic Light Controller with Safety Mode Using Rust on ESP32-S3 and Ultrasonic Sensor* merupakan sistem pengendali lampu lalu lintas adaptif berbasis *embedded system* dengan menggunakan mikrokontroler ESP32-S3, sensor ultrasonik HC-SR04, dan bahasa pemrograman Rust sebagai pengendali utama. Metode ini dikembangkan dengan menggabungkan beberapa konsep dari penelitian sebelumnya, seperti *adaptive system*, *real-time monitoring*, *fault tolerance*, serta pemrograman *embedded* yang lebih aman dan efisien. Tujuannya adalah agar sistem lampu lalu lintas dapat bekerja lebih fleksibel mengikuti kondisi jalan, tetap aman ketika terjadi gangguan, dan tetap optimal meskipun dijalankan pada perangkat dengan *resource* terbatas.

Cara kerja sistem dimulai dari sensor ultrasonik HC-SR04 yang digunakan untuk membaca jarak kendaraan secara *real-time*. Sensor akan memancarkan gelombang ultrasonik melalui pin *trigger*, kemudian menerima pantulan gelombang melalui pin *echo*. Dari waktu pantulan

tersebut, ESP32-S3 akan menghitung jarak kendaraan yang nantinya dijadikan sebagai data utama dalam menentukan kondisi lalu lintas.

Data hasil pembacaan sensor kemudian diproses menggunakan metode *adaptive timing*, yaitu sistem akan menyesuaikan durasi nyala lampu berdasarkan kondisi kendaraan di jalan. Jika kendaraan terdeteksi padat atau jarak kendaraan lebih kecil dari nilai *threshold* yang ditentukan, maka sistem akan mengaktifkan *High Traffic Safety Mode* dengan memperpanjang durasi lampu hijau menjadi sekitar 15 detik agar antrean kendaraan dapat berkurang lebih efektif. Sebaliknya, jika kondisi jalan relatif lancar atau kendaraan tidak terlalu padat, sistem akan masuk ke *Normal Traffic Mode* dengan durasi lampu hijau sekitar 5 detik sehingga waktu lampu menjadi lebih efisien.

Selain menyesuaikan durasi lampu, metode ini juga dilengkapi dengan *safety mode* sebagai langkahantisipasi jika terjadi gangguan pada sistem. Misalnya, ketika sensor gagal membaca data atau terdeteksi *error* pada pembacaan, sistem akan masuk ke mode keamanan seperti lampu kuning berkedip atau seluruh lampu berubah menjadi merah untuk sementara. Fitur ini diterapkan agar sistem tetap aman digunakan dan meminimalkan potensi konflik lalu lintas akibat kesalahan pembacaan sensor.

Konsep *resource-aware* pada metode ini diterapkan dengan cara mengoptimalkan penggunaan *resource* pada mikrokontroler, seperti penggunaan memori, waktu pemrosesan, serta penggunaan fitur-fitur penting pada ESP32-S3 sesuai dengan kebutuhan sistem. Dengan begitu, sistem dapat bekerja lebih ringan, responsif, dan tidak membebani kinerja ESP32-S3. Pembacaan sensor, pengaturan lampu, dan monitoring sistem dilakukan secara *real-time* sehingga perubahan kondisi lalu lintas dapat langsung direspons oleh sistem.

Dalam implementasinya, sistem menggunakan bahasa pemrograman Rust karena dianggap lebih aman untuk *embedded system* dibandingkan bahasa pemrograman konvensional seperti C/C++. Rust memiliki keunggulan seperti *memory safety*, *ownership system*, dan *concurrency safety* sehingga dapat mengurangi risiko *error* seperti *buffer overflow*, *null pointer*, maupun kesalahan akses memori lainnya. Pada ESP32-S3, implementasi program memanfaatkan *crate embedded-hal* dan *esp-hal* untuk mengakses GPIO, *timer*, *delay*, serta komunikasi serial. GPIO digunakan untuk mengontrol LED merah, kuning, dan hijau sebagai aktuator lampu lalu lintas, sedangkan komunikasi serial digunakan untuk menampilkan hasil pembacaan sensor dan status sistem pada *serial monitor*. Dengan menggabungkan *adaptive timing*, *safety mode*, *real-time monitoring*, serta pemrograman Rust yang lebih aman dan efisien, metode ini diharapkan dapat menghasilkan sistem lampu lalu lintas pintar yang lebih adaptif terhadap kondisi jalan, lebih aman saat terjadi gangguan, dan tetap optimal dijalankan pada perangkat *embedded* dengan *resource* terbatas.

BAB 4 Simulasi dan Hasil

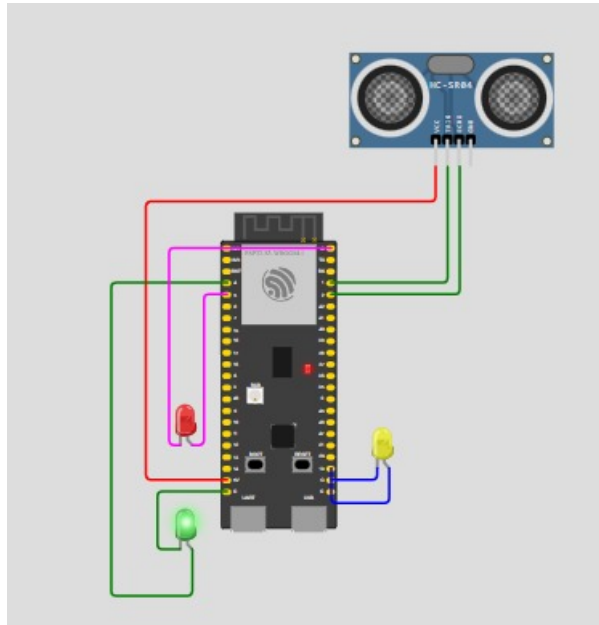


Figure 4.1: Simulasi Rangkaian

4.1 Penjelasan Rangkaian Sistem

Gambar rangkaian menunjukkan sistem *Adaptive Traffic Light Controller* menggunakan mikrokontroler ESP32-S3, sensor ultrasonik HC-SR04, dan tiga buah LED sebagai simulasi lampu lalu lintas. Sistem ini digunakan untuk mendeteksi kepadatan kendaraan berdasarkan jarak objek yang dibaca sensor ultrasonik, kemudian mengatur nyala lampu lalu lintas secara otomatis menggunakan program berbasis Rust pada ESP32-S3.

4.1.1 ESP32-S3

ESP32-S3 berfungsi sebagai mikrokontroler utama yang mengendalikan seluruh sistem. Mikrokontroler ini bertugas membaca data jarak dari sensor HC-SR04, memproses data menggunakan algoritma *adaptive timing*, kemudian mengontrol nyala LED merah, kuning, dan hijau sesuai kondisi lalu lintas. Selain itu, ESP32-S3 juga dapat menampilkan hasil pembacaan sensor pada *serial monitor*.

4.1.2 Sensor Ultrasonik HC-SR04

Sensor HC-SR04 digunakan untuk mendeteksi jarak kendaraan atau objek di depan sensor. Sensor bekerja dengan memancarkan gelombang ultrasonik melalui pin *trigger*, kemudian menerima pantulan gelombang melalui pin *echo*. Dari waktu pantulan tersebut, ESP32-S3 menghitung jarak kendaraan secara *real-time*.

Pada rangkaian:

- Pin VCC sensor terhubung ke sumber tegangan ESP32-S3
- Pin GND terhubung ke ground
- Pin Trigger dan Echo terhubung ke GPIO ESP32-S3 sebagai jalur komunikasi data sensor

4.1.3 LED Hijau

LED hijau digunakan sebagai indikator kendaraan diperbolehkan berjalan. Pada sistem *adaptive traffic light*, LED hijau akan menyala lebih lama ketika sensor mendeteksi kondisi jalan padat atau antrean kendaraan tinggi.

4.1.4 LED Kuning

LED kuning digunakan sebagai lampu transisi atau tanda hati-hati sebelum perubahan kondisi lampu lalu lintas. LED kuning biasanya menyala beberapa detik sebelum lampu berubah menjadi merah.

4.1.5 LED Merah

LED merah digunakan sebagai indikator berhenti bagi kendaraan. Ketika LED merah menyala, kendaraan harus berhenti agar arus lalu lintas dapat diatur secara bergantian.

4.1.6 Koneksi Kabel

Kabel pada rangkaian digunakan untuk menghubungkan sensor dan LED dengan GPIO ESP32-S3. Setiap LED dihubungkan ke pin output ESP32-S3 sehingga mikrokontroler dapat mengontrol nyala lampu sesuai program yang dijalankan.

Secara keseluruhan, rangkaian ini bekerja dengan cara membaca kondisi kendaraan menggunakan sensor HC-SR04, kemudian ESP32-S3 menentukan durasi nyala lampu lalu lintas berdasarkan kondisi kepadatan kendaraan secara otomatis. Sistem ini merupakan implementasi sederhana dari *smart traffic light* berbasis *embedded system* dan IoT.

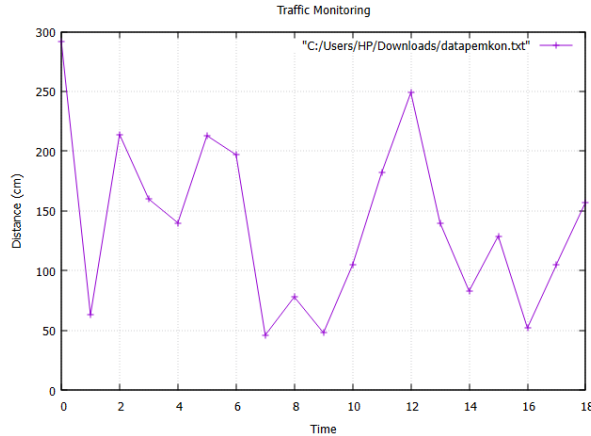


Figure 4.2: Grafik Traffic Monitoring

4.2 Analisis Grafik Traffic Monitoring

Grafik *Traffic Monitoring* menunjukkan perubahan jarak kendaraan terhadap waktu berdasarkan hasil pembacaan sensor ultrasonik HC-SR04. Pada grafik tersebut, sumbu horizontal (x -axis) menunjukkan waktu pengamatan, sedangkan sumbu vertikal (y -axis) menunjukkan jarak kendaraan yang terdeteksi oleh sensor dalam satuan sentimeter (cm). Data pada grafik digunakan untuk memonitor kondisi lalu lintas secara *real-time* sehingga sistem dapat mengetahui apakah kondisi jalan sedang padat atau lancar.

Berdasarkan grafik, nilai jarak kendaraan mengalami perubahan yang cukup dinamis pada setiap waktu pengamatan. Ketika nilai jarak yang terbaca kecil, seperti sekitar 50 cm hingga 80 cm, hal tersebut menunjukkan bahwa kendaraan berada dekat dengan sensor sehingga kondisi lalu lintas dapat dianggap padat atau terjadi antrean kendaraan. Sebaliknya, ketika nilai jarak yang terbaca lebih besar, seperti 200 cm hingga 290 cm, kondisi jalan dapat dianggap lebih lancar karena kendaraan berada lebih jauh dari sensor.

Perubahan nilai jarak pada grafik digunakan sebagai parameter utama dalam sistem *Adaptive Traffic Light Controller*. Data hasil pembacaan sensor akan diproses oleh ESP32-S3 menggunakan metode *adaptive timing*. Jika jarak kendaraan lebih kecil dari nilai *threshold* yang telah ditentukan, sistem akan menganggap kondisi lalu lintas padat dan memperpanjang durasi lampu hijau agar antrean kendaraan dapat berkurang lebih efektif. Sebaliknya, apabila jarak kendaraan lebih besar dari nilai *threshold*, sistem akan menggunakan durasi lampu normal karena kondisi jalan dianggap relatif lancar.

Dengan demikian, grafik ini menunjukkan bahwa sensor HC-SR04 mampu melakukan monitoring kondisi kendaraan secara *real-time* dan data yang diperoleh dapat digunakan untuk membantu sistem *traffic light* bekerja lebih adaptif, efisien, dan responsif terhadap perubahan kondisi lalu lintas.

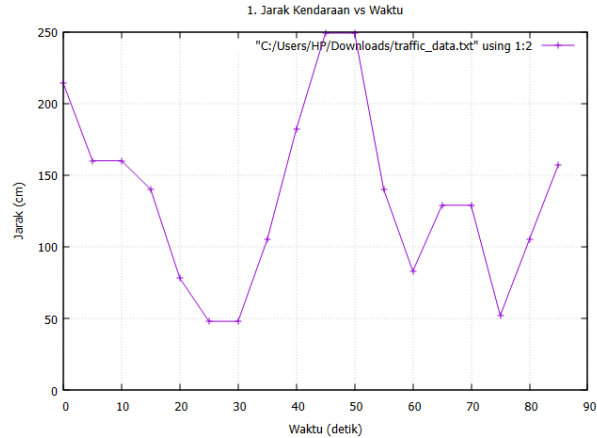


Figure 4.3: Grafik Jarak Vs Waktu

4.3 Grafik Jarak Kendaraan vs Waktu

Grafik *Jarak Kendaraan vs Waktu* menunjukkan perubahan jarak kendaraan yang dibaca oleh sensor ultrasonik HC-SR04 terhadap waktu. Sumbu horizontal menunjukkan waktu pengamatan dalam detik, sedangkan sumbu vertikal menunjukkan jarak kendaraan dalam satuan sentimeter (cm).

Pada grafik terlihat bahwa nilai jarak kendaraan berubah-ubah secara dinamis. Ketika nilai jarak menurun hingga sekitar 50 cm, kondisi lalu lintas dapat dianggap padat karena kendaraan berada dekat dengan sensor. Sebaliknya, ketika nilai jarak meningkat hingga sekitar 250 cm, kondisi jalan dianggap lebih lancar karena kendaraan berada lebih jauh dari sensor.

Data jarak ini digunakan sebagai parameter utama dalam sistem *adaptive traffic light*. ESP32-S3 akan memproses data tersebut untuk menentukan durasi nyala lampu hijau secara otomatis. Dengan adanya monitoring jarak secara *real-time*, sistem dapat menyesuaikan kondisi lalu lintas secara lebih responsif dibandingkan sistem *traffic light* konvensional berbasis timer tetap.

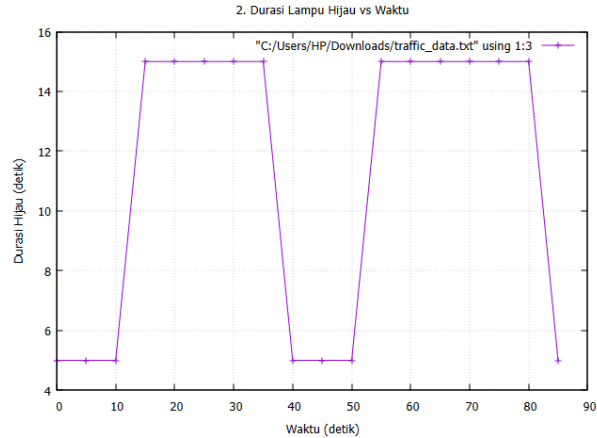


Figure 4.4: Grafik Durasi Lampu Vs Waktu

4.4 Grafik Durasi Lampu Hijau vs Waktu

Grafik *Durasi Lampu Hijau vs Waktu* menunjukkan perubahan durasi nyala lampu hijau terhadap waktu berdasarkan kondisi kendaraan yang dibaca sensor HC-SR04. Sumbu horizontal menunjukkan waktu dalam detik, sedangkan sumbu vertikal menunjukkan durasi lampu hijau dalam satuan detik.

Berdasarkan grafik, durasi lampu hijau berubah antara 5 detik dan 15 detik sesuai kondisi lalu lintas. Ketika sensor mendeteksi kendaraan dalam kondisi padat atau jarak kendaraan berada di bawah nilai *threshold*, sistem akan memperpanjang durasi lampu hijau menjadi sekitar 15 detik agar antrean kendaraan dapat berkurang lebih efektif. Sebaliknya, ketika kondisi jalan relatif lancar, sistem akan menggunakan durasi lampu hijau normal sekitar 5 detik untuk meningkatkan efisiensi pengaturan lalu lintas.

Grafik ini menunjukkan bahwa metode *adaptive timing* berhasil diterapkan pada sistem. Sistem mampu menyesuaikan durasi lampu hijau secara otomatis berdasarkan kondisi kendaraan secara *real-time* sehingga pengaturan lalu lintas menjadi lebih fleksibel, adaptif, dan efisien.

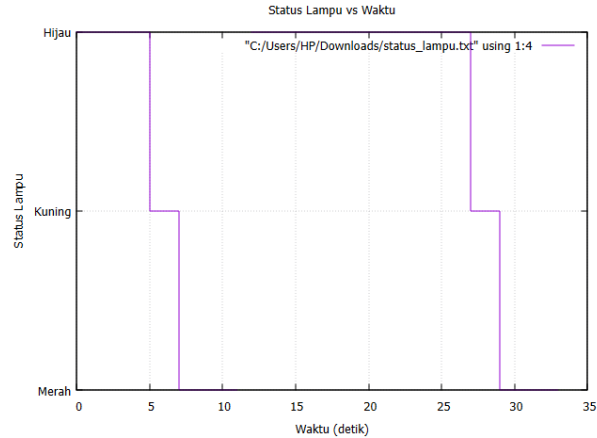


Figure 4.5: Grafik Status Lampu Vs Waktu

4.5 Grafik Status Lampu vs Waktu

Grafik *Status Lampu vs Waktu* menunjukkan perubahan kondisi lampu lalu lintas terhadap waktu pada sistem *Adaptive Traffic Light Controller*. Pada sumbu horizontal (x -axis) ditunjukkan waktu dalam satuan detik, sedangkan sumbu vertikal (y -axis) menunjukkan status lampu lalu lintas yang terdiri dari lampu hijau, kuning, dan merah.

Berdasarkan grafik, sistem memulai kondisi dengan lampu hijau yang menyala selama beberapa detik sebagai tanda kendaraan diperbolehkan berjalan. Setelah itu, sistem berpindah ke lampu kuning sebagai fase transisi sebelum akhirnya berubah menjadi lampu merah untuk menghentikan kendaraan sementara. Setelah siklus selesai, sistem kembali mengulang pola yang sama secara otomatis.

Grafik ini menunjukkan bahwa sistem *traffic light* mampu bekerja secara berurutan dan otomatis sesuai algoritma *adaptive timing* yang telah dirancang. Perubahan status lampu dilakukan secara *real-time* berdasarkan hasil pembacaan sensor HC-SR04 sehingga pengaturan lalu lintas menjadi lebih adaptif dan efisien.

BAB 5 Keunggulan Metode

Metode *Adaptive Resource-Aware Traffic Light Controller with Safety Mode Using Rust on ESP32-S3 and Ultrasonic Sensor* memiliki beberapa keunggulan dibandingkan metode pada jurnal-jurnal sebelumnya. Pada beberapa penelitian sebelumnya, sistem lampu lalu lintas masih menggunakan sistem *timer tetap (fixed timer)* sehingga durasi lampu tidak dapat menyesuaikan kondisi kendaraan di jalan. Akibatnya, saat jalan sedang kosong maupun padat, waktu lampu tetap sama sehingga kurang efisien.

Pada metode yang diusulkan, sistem menggunakan konsep *adaptive timing* di mana durasi lampu lalu lintas dapat berubah secara otomatis berdasarkan hasil pembacaan sensor ultrasonik secara *real-time*. Ketika antrean kendaraan terdeteksi padat, lampu hijau akan menyala lebih lama agar kemacetan dapat berkurang lebih cepat. Sebaliknya, ketika jalan relatif lancar, durasi lampu hijau dibuat lebih singkat sehingga pengaturan lalu lintas menjadi lebih efisien dan responsif terhadap kondisi jalan.

Selain itu, metode ini juga menerapkan konsep *resource-aware system* dengan mengoptimalkan penggunaan memori, waktu pemrosesan, dan fitur-fitur pada ESP32-S3 sesuai kebutuhan sistem. Dengan begitu, sistem dapat berjalan lebih ringan, stabil, dan tidak membebani kinerja mikrokontroler meskipun dijalankan pada perangkat *embedded* dengan *resource* terbatas.

Dari sisi keamanan sistem, metode ini memiliki keunggulan karena dilengkapi dengan *safety mode* dan *fault recovery mechanism*. Jika sensor mengalami *error* atau gagal membaca data, sistem akan otomatis masuk ke mode keamanan seperti lampu kuning berkedip atau seluruh lampu berubah menjadi merah sementara. Fitur ini membuat sistem lebih aman dibandingkan metode sebelumnya yang belum memiliki mekanisme penanganan *error* secara otomatis.

Keunggulan lainnya terletak pada penggunaan bahasa pemrograman Rust sebagai pengendali utama sistem *embedded*. Dibandingkan bahasa pemrograman konvensional seperti C/C++, Rust memiliki fitur *memory safety* dan *ownership system* yang membantu mengurangi risiko *error* seperti *buffer overflow*, *null pointer*, dan kesalahan akses memori lainnya. Dengan begitu, sistem menjadi lebih aman dan stabil untuk digunakan pada aplikasi *embedded* dan IoT modern.

Kontribusi orisinalitas dari metode ini yaitu menggabungkan beberapa konsep *future work* yang sebelumnya belum banyak diterapkan secara bersamaan dalam satu sistem lampu lalu lintas pintar. Metode ini mengombinasikan *adaptive traffic monitoring*, *resource-aware processing*, *safety mode*, *fault tolerance system*, serta pemrograman *embedded* berbasis Rust pada ESP32-S3 menjadi satu sistem yang terintegrasi.

Selain itu, kombinasi penggunaan sensor ultrasonik sebagai pendeteksi kepadatan kendaraan dengan pendekatan pemrograman *embedded* berbasis Rust masih jarang digunakan pada penelitian sistem lampu lalu lintas sebelumnya. Oleh karena itu, metode ini diharapkan dapat menjadi pendekatan baru dalam pengembangan *intelligent transportation system* berbasis *embedded system* dan IoT yang lebih adaptif, aman, dan efisien.

BAB 6 Dokumentasi GitHub dan Publikasi LinkedIn

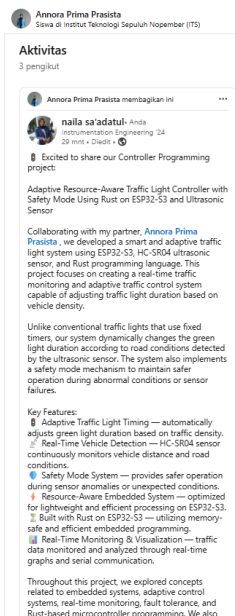
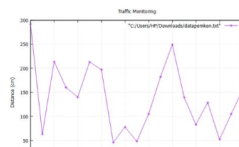
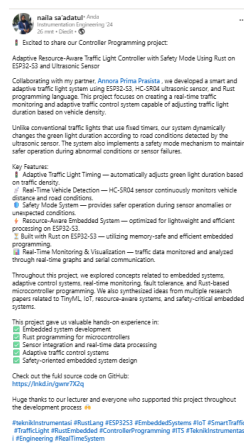
6.1 Repository GitHub

Source code, simulasi, dan implementasi metode dapat diakses melalui repository GitHub berikut:

<https://github.com/annorapr/adaptive-traffic-light-rust>

Repository tersebut berisi implementasi sistem *Adaptive Resource-Aware Traffic Light Controller with Safety Mode Using Rust on ESP32-S3 and Ultrasonic Sensor* menggunakan bahasa pemrograman Rust pada ESP32-S3 serta simulasi monitoring lalu lintas berbasis sensor ultrasonik HC-SR04.

6.2 LinkedIn



BAB 7 Kesimpulan

Berdasarkan hasil perancangan dan simulasi yang telah dilakukan, sistem *Adaptive Resource-Aware Traffic Light Controller with Safety Mode Using Rust on ESP32-S3 and Ultrasonic Sensor* berhasil dirancang sebagai sistem lampu lalu lintas adaptif berbasis *embedded system*. Sistem mampu mendeteksi kondisi kepadatan kendaraan secara *real-time* menggunakan sensor ultrasonik HC-SR04 dan memproses data tersebut menggunakan mikrokontroler ESP32-S3 berbasis bahasa pemrograman Rust.

Metode *adaptive timing* yang diterapkan mampu menyesuaikan durasi lampu hijau berdasarkan kondisi lalu lintas. Ketika kendaraan terdeteksi padat, sistem secara otomatis memperpanjang durasi lampu hijau agar antrean kendaraan dapat berkurang lebih efektif. Sebaliknya, ketika kondisi jalan relatif lancar, sistem menggunakan durasi lampu yang lebih singkat sehingga pengaturan lalu lintas menjadi lebih efisien.

Selain itu, penerapan *safety mode* pada sistem dapat meningkatkan keamanan operasional ketika terjadi gangguan pembacaan sensor atau kesalahan sistem. Dengan adanya mode keamanan seperti lampu kuning berkedip atau seluruh lampu merah sementara, sistem menjadi lebih aman dan mampu meminimalkan potensi konflik lalu lintas.

Penggunaan bahasa pemrograman Rust juga memberikan keunggulan pada sisi *memory safety*, efisiensi penggunaan *resource*, serta stabilitas sistem dibandingkan bahasa pemrograman *embedded* konvensional. Dengan demikian, metode yang diusulkan diharapkan dapat menjadi salah satu solusi pengembangan *intelligent transportation system* berbasis IoT dan *embedded system* yang lebih adaptif, aman, efisien, dan responsif terhadap kondisi lalu lintas secara *real-time*.

Bibliography

- [1] M. Carneiros et al., “MicroFlow: An Efficient Rust-Based Inference Engine for TinyML,” 2025.
- [2] A. Sharma et al., “Rust for Embedded Systems: Current State and Open Problems,” 2024.
- [3] J. Chaoraingern and A. Numsomran, “Embedded Sensor Data Fusion and TinyML for Real-Time Remaining Useful Life Estimation of UAV Li Polymer Batteries,” 2025.
- [4] N. Witt et al., “Energy-Efficient AI on the Edge,” 2024.
- [5] N. Gupta et al., “A Hard Real-Time Kernel for CPIoT Systems With Safety Features in Rust,” 2025.
- [6] A. Sardaryan et al., “An Accurate Real-Time Object Tracking Method for Resource-Constrained Devices,” 2024.
- [7] P.-E. Novac et al., “Quantization and Deployment of Deep Neural Networks on Microcontrollers,” 2021.
- [8] M. A. Solouki et al., “Dependability in Embedded Systems: A Survey of Fault Tolerance Methods and Software-Based Mitigation Techniques,” 2024.
- [9] L. A. P. Ortiz et al., “Advancing TinyML in IoT: A Holistic System-Level Perspective for Resource-Constrained AI,” 2025.
- [10] E. Baccour et al., “Pervasive AI for IoT Applications: A Survey on Resource-Efficient Distributed Artificial Intelligence,” 2022.
- [11] A. N. M. Rafee et al., “Composite Human Activity Recognition Utilizing Knowledge Distillation and Sensor Fusion Focusing on Resource Constrained Microcontrollers,” 2026.
- [12] Z. Wen et al., “ESP-Fi HAR: A Low-Power WiFi CSI Dataset for Ad-Hoc IoT Human Activity Recognition,” 2026.
- [13] S. Essahraoui and I. Lamaakal, “A Comprehensive Survey of TinyML-Based Biometric Recognition for IoT Edge Devices,” 2026.
- [14] A. N. M. Rafee et al., “Composite Human Activity Recognition Utilizing Knowledge Distillation and Sensor Fusion Focusing on Resource Constrained Microcontrollers,” 2026.
- [15] P.-E. Novac et al., “Quantization and Deployment of Deep Neural Networks on Microcontrollers,” 2021.